

Перевірка багатопотоковості в лабораторній роботі №14

Об'єктно-орієнтоване програмування.

Java Базова багатопотоковість у Java

Студент: Колісніченко Артем Олександрович

Група: КН-924в

Викладач: Володимир Миколайович Савченко

Рік: 2026

1. Створення потоків у програмі

У лабораторній роботі реалізовано предметну область «Медіатека». Для демонстрації багатопотоковості використано три незалежні завдання: сканування каталогу, сортування фільмів і генерація рекомендацій.

У роботі використано два способи створення потоків. Перший спосіб — клас `CatalogScanThread`, який наслідує `Thread`. Він відповідає за сканування медіакаталогу.

Другий спосіб — класи `SortTask` і `RecommendationTask`, які реалізують інтерфейс `Runnable`. Клас `SortTask` відповідає за сортування фільмів за рейтингом, а клас `RecommendationTask` — за формування списку рекомендованих фільмів.

У класі `Main` створюються три робочі потоки:

```
CatalogScanThread scanThread = new CatalogScanThread(catalog, service);
```

```
SortTask sortTask = new SortTask(catalog, service);
```

```
Thread sortThread = new Thread(sortTask, "SortThread");
```

```
RecommendationTask recommendationTask = new  
RecommendationTask(catalog, service);
```

```
Thread recommendationThread = new Thread(recommendationTask,  
"RecommendationThread");
```

Тобто в програмі є три робочі потоки: `CatalogScanThread`, `SortThread` і `RecommendationThread`.

2. Запуск потоків

Потоки запускаються методом `start()`:

```
scanThread.start();  
sortThread.start();  
recommendationThread.start();
```

Саме метод `start()` створює окремий потік виконання. Якщо викликати метод `run()` напямую, код буде виконуватися послідовно в потоці `main`, і це не буде реальною багатопотоковістю.

Після запуску використовується метод `join()`:

```
scanThread.join();  
sortThread.join();  
recommendationThread.join();
```

Метод `join()` потрібен для того, щоб головний потік `main` дочекався завершення всіх робочих потоків. Тільки після цього програма виводить фінальні результати.

3. Перевірка станів потоків у дебагері

Для перевірки виконання потоків у дебагері програма була зупинена після викликів `start()` на рядку з `join()`.

Через метод `getState()` було перевірено стан кожного потоку:

```
"scan=" + scanThread.getState()  
    + ", sort=" + sortThread.getState()  
    + ", recommendation=" + recommendationThread.getState()
```

У дебагері було отримано результат:

```
scan=RUNNABLE, sort=RUNNABLE, recommendation=RUNNABLE
```

Це означає, що всі три потоки вже запущені та передані JVM для виконання. Стан `RUNNABLE` показує, що потік готовий виконуватись або вже виконується операційною системою.

Також можливий стан `TIMED_WAITING`, якщо потік у цей момент знаходиться в методі `Thread.sleep()`. Це також є нормальним результатом, тому що потік уже запущений і тимчасово очікує.



Рисунок 1 — Перевірка станів трьох потоків після виклику `start()`. Потіки мають стан `RUNNABLE`, що підтверджує їх реальний запуск.

4. Виконання `RecommendationTask` в окремому потоці

У дебагері було встановлено breakpoint у методі `run()` класу `RecommendationTask`.

На скриншоті видно, що поточний потік має ім'я:

`RecommendationThread`

Це підтверджує, що код методу `run()` виконується не в головному потоці `main`, а в окремому робочому потоці `RecommendationThread`.

Цей потік формує список рекомендованих фільмів через сервіс `MediaService`.

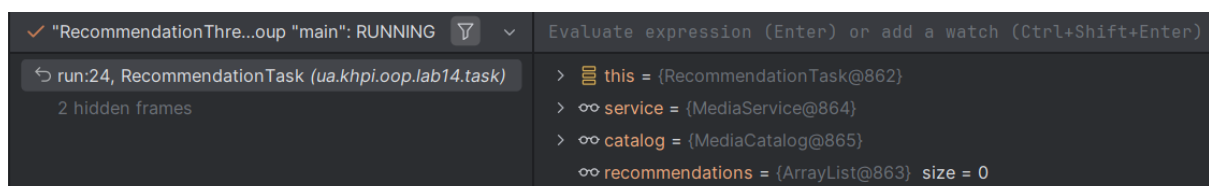


Рисунок 2 — Виконання методу `run()` класу `RecommendationTask` у потоці `RecommendationThread`.

5. Виконання CatalogScanThread в окремому потоці

Також breakpoint був встановлений у методі run() класу CatalogScanThread.

У дебагері видно, що виконання відбувається в потоці:

CatalogScanThread

Цей потік виконує сканування медіакаталогу: перевіряє кількість фільмів, кількість подкастів і загальну кількість елементів у каталозі.

Клас CatalogScanThread є прикладом створення потоку через наслідування від класу Thread.

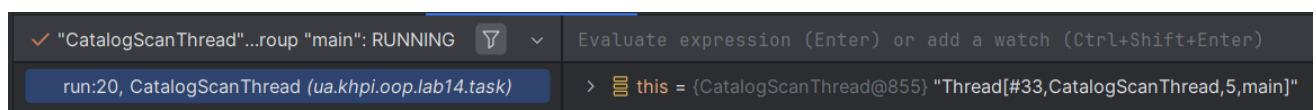


Рисунок 3 — Виконання методу run() класу CatalogScanThread в окремому потоці CatalogScanThread.

6. Виконання SortTask в окремому потоці

Додатково можна перевірити виконання методу run() класу SortTask.

У дебагері поточний потік повинен мати ім'я:

SortThread

Це підтверджує, що сортування фільмів за рейтингом також виконується в окремому потоці, а не в головному потоці main.

Клас SortTask реалізує інтерфейс Runnable, а сам потік створюється через об'єкт Thread.

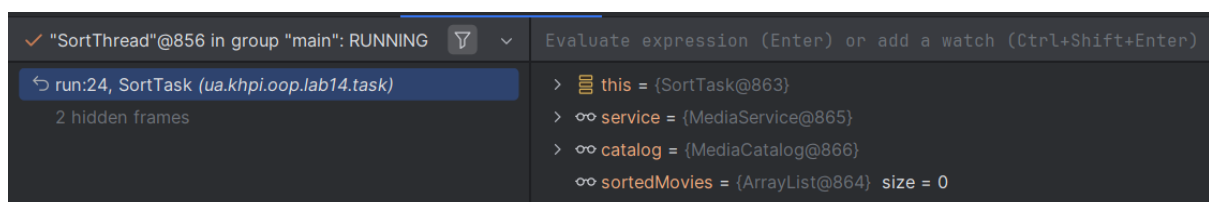


Рисунок 4 — Виконання методу run() класу SortTask у потоці SortThread.

7. Висновок по демонстрації

У результаті було показано, що в програмі працює багатопотоковість. У класі Main створюються та запускаються три окремі потоки через метод `start()`.

У дебагері видно, що ці потоки мають стан `RUNNABLE` і заходять у власні методи `run()`. Потік `CatalogScanThread` виконує сканування каталогу, потік `SortThread` виконує сортування фільмів, а потік `RecommendationThread` формує рекомендації.

Метод `join()` використовується для того, щоб головний потік `main` дочекався завершення всіх робочих потоків. Завдяки цьому фінальні результати виводяться тільки після завершення всієї роботи.

Таким чином, у лабораторній роботі продемонстровано базову багатопотоковість Java: створення потоків через `Thread` і `Runnable`, запуск через `start()`, імітацію роботи через `sleep()` та очікування завершення через `join()`.